

Integração MQTT — Home Assistant

Guia técnico para desenvolvedores Android

1. Visão Geral

O app publica dados do veículo em um broker MQTT. O Home Assistant consome esses dados via integração nativa de MQTT e cria sensores automaticamente através do protocolo **MQTT Discovery** — sem nenhuma configuração manual no HA.

Fluxo:



2. Dependências

Adicionar no **app/build.gradle.kts**:

```
implementation("org.eclipse.paho:org.eclipse.paho.client.mqttv3:1.2.5")
implementation("org.slf4j:slf4j-nop:1.7.36") // obrigatório – veja seção 4
```

3. Permissões

Adicionar no **AndroidManifest.xml**, dentro de <manifest>:

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

4. Problema Crítico: Logger do Paho no Android

■■ Este é o principal obstáculo. Sem esse fix, o app falha ao conectar com a mensagem "Error locating the logging class" e nunca estabelece conexão MQTT.

O Paho tenta carregar seu logger interno via `Class.forName().newInstance()`. Quando isso falha no Android, a exceção vaza do construtor do `MqttClient` e impede qualquer conexão.

Fix: SLF4J versão correta

Usar obrigatoriamente a versão **1.7.36**, não 2.x:

```
implementation("org.slf4j:slf4j-nop:1.7.36") // NÃO use 2.x
```

■■ O Paho 1.2.5 usa a API SLF4J 1.x. A versão 2.x mudou o mecanismo de binding (`ServiceLoader` vs. `classpath scan`) e é incompatível.

Fix adicional: PahoNoOpLogger

Se o erro persistir, criar uma classe que implementa a interface `org.eclipse.paho.client.mqttv3.logging.Logger` com todos os métodos vazios e registrá-la **antes** de instanciar qualquer `MqttClient`:

```
// Registrar no início da inicialização MQTT:
org.eclipse.paho.client.mqttv3.logging.LoggerFactory
```

```

        .setLogger(PahoNoOpLogger::class.java.name)

// A classe PahoNoOpLogger implementa todos os métodos da interface Logger
// do Paho com corpos vazios (no-op). O compilador Kotlin indicará exatamente
// quais assinaturas implementar – seguir os erros de compilação.

```

5. Conectar ao Broker

Toda comunicação deve ocorrer em uma thread separada (nunca na main thread):

```

private val executor = Executors.newSingleThreadExecutor()

fun connect() = executor.submit {
    val client = MqttClient(
        "tcp://SEU_IP:1883",
        "meu_app_${System.currentTimeMillis() % 10000}", // ID único
        MemoryPersistence()
    )
    val opts = MqttConnectOptions().apply {
        connectionTimeout      = 20
        keepAliveInterval      = 30
        isCleanSession          = true
        isAutomaticReconnect    = false // gerenciar manualmente
        userName = "usuario"
        password = "senha".toCharArray()
        // LWT: marca o dispositivo como offline se a conexão cair
        setWill("prefixo/status", "offline".toByteArray(), 1, true)
    }
    client.setCallback(object : MqttCallback {
        override fun connectionLost(cause: Throwable?) {
            scheduleReconnect() // tentar novamente após delay
        }
        override fun messageArrived(t: String, m: MqttMessage) {}
        override fun deliveryComplete(t: IMqttDeliveryToken) {}
    })
    client.connect(opts)
    client.publish("prefixo/status",
        MqttMessage("online".toByteArray())).apply { qos = 1; isRetained = true }
}

```

■ Usar CLIENT_ID com sufixo de timestamp evita conflito ao reiniciar o app rápido.

6. Home Assistant — MQTT Discovery

Publicar uma mensagem de configuração para cada sensor no tópico de discovery. O HA processa automaticamente e cria os sensores no dispositivo.

Tópico do discovery:

homeassistant/sensor/{unique_id}/config

Payload JSON (publicar com **retain=true** e **QoS 1**):

```

{
  "name": "Nome do Sensor",
  "state_topic": "meu_app/topico/valor",
  "unit_of_measurement": "km/L",
  "unique_id": "meu_app_sensor_id",
  "device_class": "distance", // opcional
}

```

```

"icon": "mdi:gas-station",          // opcional
"device": {
    "identifiers": ["meu_app"],
    "name": "Nome do Dispositivo",
    "model": "Modelo do Carro",
    "manufacturer": "Fabricante"
}
}

```

■ Todos os sensores que compartilham o mesmo campo 'device.identifiers' são agrupados automaticamente como um único dispositivo no HA.

7. Publicar Valores

Após o discovery, publicar o valor atual no state_topic:

```

// Valor simples (string)
client.publish("meu_app/topico/valor", "18.5".toByteArray(), 0, false)

```

```

// Evento importante (não perder, ficar salvo no broker)
client.publish("meu_app/viagem/concluida",
    payload.toByteArray(), 1, true) // QoS 1, retain=true

```

Para sensores com múltiplos atributos (ex: dados completos de uma viagem):

```

// No discovery, adicionar:
"value_template": "{{ value_json.distancia_km }}",
"json_attributes_topic": "meu_app/viagem/concluida"

```

```

// Publicar JSON com todos os dados:
val json = """{"distancia_km": 48.2, "combustivel_l": 3.1, "nome": "Goiânia→Catalão"}"""
client.publish("meu_app/viagem/concluida", json.toByteArray(), 1, true)

```

8. Reconexão Automática

■ Usar `isAutomaticReconnect=false` e gerenciar manualmente. O reconnect automático do Paho tem comportamento imprevisível no Android.

```

private val isReconnecting = AtomicBoolean(false)

private fun scheduleReconnect() {
    if (!enabled || isReconnecting.getAndSet(true)) return
    executor.submit {
        Thread.sleep(15_000) // aguardar 15s antes de tentar
        isReconnecting.set(false)
        if (enabled) connect()
    }
}

```

9. Fila Offline — Dados Nunca Perdidos

Se o app perder conexão, enfileirar os dados e enviar tudo ao reconectar. Dois níveis de fila recomendados:

Tipo	Comportamento	Motivo
Snapshots periódicos	Manter apenas os últimos N (ex: 50)	Apenas o mais recente importa para o estado atual no HA
Eventos de viagem concluída	Nunca descartar, re-enfileirar em falha	Dado histórico — perder é irreversível

10. Checklist de Implementação

- Dependências adicionadas (Paho 1.2.5 + SLF4J 1.7.36)
- Permissões INTERNET e ACCESS_NETWORK_STATE no Manifest
- PahoNoOpLogger criado e registrado antes do primeiro MqttClient
- Conexão executada em thread separada (ExecutorService)
- LWT configurado: dispositivo aparece offline no HA se app cair
- Discovery publicado uma vez após cada conexão (retain=true, QoS 1)
- Valores periódicos publicados no intervalo configurado
- Reconexão manual com delay de 15s implementada
- Fila offline implementada — dados enviados ao reconectar
- CLIENT_ID único por sessão para evitar conflito de reconnect